

XS WALLET

Go Core + Atomic Swaps Desktop

■ SELF-CUSTODY • ■ LIGHTNING • ■ TAPROOT • ■ GO CORE

Documentação Técnica v0.2.0

20 de Janeiro 2026

Equipe XS Wallet

■ Sumário Executivo

Visão geral do projeto XS Wallet v2

XS Wallet é uma aplicação desktop self-custody que permite atomic swaps entre Bitcoin, Liquid e Lightning Network usando Taproot e **boltz-backend (self-hosted)** como swap orchestrator com **nossa liquidez**. O sistema mantém verdadeira auto-custódia enquanto fornece swaps cross-chain e cross-layer.

Características Principais

- Go Core (xscore): Backend autoritativo em Go com gRPC
- Carteira HD compatível com BIP39/32/84/85
- Suporte Multi-Chain: Bitcoin, Liquid, Lightning
- Atomic Swaps via boltz-backend (compatível API v2)
- Boltz Client production-ready com retry/backoff
- WebSocket single-writer + gap-recovery
- Integração Taproot com MuSig2 + fallback script-path
- Auto-custódia com Argon2id + AES-256-GCM

Status do Projeto (40%)

- ■ Go Core (xscore) - Implementado
- ■ Boltz Client HTTP - Production-Ready
- ■ WebSocket Client - Production-Ready
- ■ Status Normalization - Implementado
- ■ Swap Engine Base - Implementado
- ■ LND Adapter - Próximo Sprint
- ■ Liquid Adapter - Próximo Sprint

- ■ Frontend Electron - Backlog

■ ■ ■ Arquitetura do Sistema

Go Core + boltz-backend self-hosted

O XS Wallet é estruturado com **Go Core** como backend autoritativo, comunicando via **gRPC** com o frontend Electron. O swap orchestrator é o **boltz-backend self-hosted** com nossa liquidez. As interfaces RPC primárias são **LND (gRPC)** e **elements (JSON-RPC)**, com bitcoind como dependência do runtime.

Componentes Principais

- Go Core (xscore): Swap Engine, State Machine, CAS/Idempotência
- Boltz Client: HTTP + Retry/Backoff, WebSocket Single-Writer
- Adapters: LND (gRPC), elements (JSON-RPC)
- boltz-backend: Self-hosted, nossa liquidez, REST compatível v2
- Frontend: Electron + React (migrar de BRLN devdash)

2 Interfaces RPC Primárias (conforme feedback CTO): LND para Lightning (invoices, payments, channels) via gRPC, e elements para Liquid (addresses, transactions, blinding) via JSON-RPC. Bitcoind é dependência do runtime, não interface primária.

■ Boltz Client - Production-Ready

Implementação robusta e resiliente

Features Implementadas

- HTTP Client com exponential backoff
- Rate limit handling (Retry-After header)
- Resource leak prevention (body fechado em closure)
- WebSocket com padrão Single-Writer (concorrência segura)
- Gap Recovery automático via REST após reconexão
- Status normalization table-driven por tipo de swap
- Tratamento de MinerFees polimórfico (number OU object)
- Erros tipados (ErrSwapNotFound, ErrPairHashMismatch, etc)

Arquivos Implementados (core/internal/boltz/)

- errors.go - Erros tipados de domínio
- types.go - Structs API v2 com MinerFeesAny polimórfico
- client.go - HTTP Client com retry/backoff/Retry-After
- ws.go - WebSocket single-writer + gap-recovery
- status.go - Status normalization table-driven
- boltz.go - Provider interface implementation
- *_test.go - Testes unitários (100% passing)

■ Banco de Dados

SQLite com WAL mode para concorrência

Tabela	Propósito	Chave Primária
swaps	Estado autoritativo (CAS)	id (TEXT)
swap_events	Trilha de auditoria	seq (AUTOINCREMENT)
swap_ops	Ledger idempotência	(swap_id, op_key)
utxo_reservations	Anti double-spend	(chain, txid, vout)
ln_reservations	Anti duplicação LN	payment_hash_hex
app_config	Configurações	key

Diferencial Elogiado pelo CTO: "Isso aqui é coisa de gente cuidadosa" - refere-se ao modelo autoritativo de estado com **locked_intent** como snapshot imutável, **CAS otimista** com version e update atômico, e tabelas de **idempotência** (swap_ops, utxo_reservations, ln_reservations). Sistema que não perde dinheiro em restart, não duplica broadcast, não paga LN duas vezes.

■ Stack Tecnológico

Tecnologias modernas para 2026

Componente	Tecnologia	Versão
Backend	Go	1.25+
Comunicação	gRPC	1.60+
Frontend	React + Vite	18 + 5
Desktop	Electron	28+
Database	go-sqlite3	1.14+
WebSocket	gorilla/websocket	1.5+
Bitcoin	btcsuite/btcd	0.24+
Crypto	x/crypto (Argon2id)	latest

■ Roadmap de Implementação

Sprint Atual + Próximos Passos

Sprint Atual: Spike boltz-backend

- Setup boltz-backend docker (regtest)
- Conectar LND (gRPC)
- Conectar elementsd (JSON-RPC)
- Testar 3 tipos de swap (Submarine/Reverse/Chain)
- Documentar responsibility split: Wallet vs boltz-backend

Fase	Status	Entregas
Boltz Client	■ Concluída	HTTP, WebSocket, Status Normalization
Spike boltz-backend	■ Atual	Responsibility Split
Adapters	■ Próximo	LND gRPC, elementsd JSON-RPC
Wiring E2E	■ Backlog	xscore → boltz-backend, Testes
Frontend	■ Backlog	Migrar BRLN devdash, Electron

Roadmap Pós-MVP

- v1.1: Hardware wallet, SQLCipher, Multi-wallet
- v1.2: Coinjoin, Payjoin, LNURL
- v2.0: EVM Module (MetaMask + LI.FI) - isolado do core UTXO

XS Wallet v0.2.0 • 20 Janeiro 2026 • Fase 1 Concluída (40%)